Scientific Machine Learning Final Project

Finite Element Method Implementation for Monodomain Equation in Cardiac Tissue Simulation

Paolo Deidda, Raffaele Perri

June 2025

Contents

1 Theoretical Formulations

1.1 IMEX Time Integration Scheme

To solve the monodomain equation efficiently, we use a first-order Implicit-Explicit (IMEX) scheme that treats the diffusion term implicitly and the nonlinear reaction term explicitly. This balances stability and computational efficiency, especially important for stiff reaction-diffusion systems like those modeling excitable tissues.
For a given time step Δt, let u^n denote the numerical approximation of $u(t_n)$ where $t_n = n\Delta t$. The first-order IMEX scheme can be written as:

$$\frac{u^{n+1} - u^n}{\Delta t} - \nabla \cdot \Sigma \nabla u^{n+1} + f(u^n) = 0 \tag{1}$$

Rearranging this equation to isolate the unknown u^{n+1}, we obtain:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \tag{2}$$

This can be written in operator form as:

$$(I - \Delta t \mathcal{L}) u^{n+1} = u^n - \Delta t f(u^n) \tag{3}$$

where I is the identity operator and $\mathcal{L}$ is the diffusion operator defined by $\mathcal{L}u = \nabla \cdot \Sigma \nabla u$. The implicit treatment of diffusion ensures unconditional stability in space, while the explicit treatment of the reaction term avoids nonlinear solvers at each step.

1.2 Weak Formulation

Starting from the IMEX scheme:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \tag{4}$$

Multiplying by the test function v and integrating over Ω:

$$\int_\Omega \left(u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1}\right) v \, d\Omega = \int_\Omega \left(u^n - \Delta t f(u^n)\right) v \, d\Omega \tag{5}$$

The key step in the derivation is the application of integration by parts to the diffusion term. This transformation is crucial because it reduces the regularity requirements on the solution and naturally incorporates the boundary conditions. Applying integration by parts to the second term on the left-hand side:

$$\int_\Omega \nabla \cdot \Sigma \nabla u^{n+1} v \, d\Omega = \int_{\partial\Omega} \Sigma \frac{\partial u^{n+1}}{\partial \mathbf{n}} v \, d\Gamma - \int_\Omega \Sigma \nabla u^{n+1} \cdot \nabla v \, d\Omega \tag{6}$$

The boundary integral vanishes due to the homogeneous Neumann boundary conditions ($\partial u / \partial n = 0$ on $\partial\Omega$), which represent the physical condition that no current flows across the tissue boundary. This leads to the weak formulation:
Find $u^{n+1} \in H^1(\Omega)$ such that for all $v \in H^1(\Omega)$:

$$\int_\Omega u^{n+1} v \, d\Omega + \Delta t \int_\Omega \Sigma \nabla u^{n+1} \cdot \nabla v \, d\Omega = \int_\Omega u^n v \, d\Omega - \Delta t \int_\Omega f(u^n) v \, d\Omega \tag{7}$$

This weak formulation can be expressed in the standard bilinear form $a(u^{n+1}, v) = L^n(v)$, where:

$$a(u, v) = \int_\Omega uv \, d\Omega + \Delta t \int_\Omega \Sigma \nabla u \cdot \nabla v \, d\Omega \tag{8}$$

$$L^n(v) = \int_\Omega u^n v \, d\Omega - \Delta t \int_\Omega f(u^n) v \, d\Omega \tag{9}$$

This weak formulation is symmetric, coercive, and continuous—ensuring existence and uniqueness of the solution via the Lax-Milgram theorem.

1.3 Finite Element Discretization

The finite element space is defined as:

$$V_h = \{v_h \in C^0(\Omega) : v_h|_K \in Q_1(K) \text{ for all elements } K\} \tag{10}$$

where $Q_1(K)$ denotes the space of bilinear functions on element K.

The finite element approximation is expressed as:

$$u_h^{n+1}(x) = \sum_{i=1}^{n_v} U_i^{n+1} \phi_i(x) \tag{11}$$

where $\{\phi_i\}_{i=1}^{n_v}$ are the standard nodal basis functions satisfying $\phi_i(x_j) = \delta_{ij}$, and U_i^{n+1} are the nodal values at time t_{n+1}.

Substituting this approximation into the weak formulation and choosing $v = \phi_j$ for $j = 1, \ldots, n_v$, we obtain the linear system:

$$\mathbf{A}\mathbf{U}^{n+1} = \mathbf{b}^n \tag{12}$$

where:

- $\mathbf{U}^{n+1} = [U_1^{n+1}, U_2^{n+1}, \ldots, U_{n_v}^{n+1}]^T$ is the solution vector
- $\mathbf{A}$ is the system matrix
- $\mathbf{b}^n$ is the right-hand side vector

The system matrix $\mathbf{A}$ can be decomposed as:

$$\mathbf{A} = \mathbf{M} + \Delta t \mathbf{K} \tag{13}$$

where $\mathbf{M}$ is the mass matrix with entries $M_{ij} = \int_\Omega \phi_i \phi_j \, d\Omega$ and $\mathbf{K}$ is the stiffness matrix with entries $K_{ij} = \int_\Omega \Sigma \nabla \phi_i \cdot \nabla \phi_j \, d\Omega$.

The right-hand side vector is given by:

$$\mathbf{b}^n = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \tag{14}$$

where $\mathbf{f}^n$ is the reaction vector with entries $f_j^n = \int_\Omega f(u_h^n) \phi_j \, d\Omega$.

The final linear system at each time step is:

$$(\mathbf{M} + \Delta t \mathbf{K})\mathbf{U}^{n+1} = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \tag{15}$$

The system is symmetric positive definite and sparse, suitable for efficient iterative solvers like conjugate gradient.

1.4 Modified assembleDiffusion Function

We modify assembleDiffusion to support element-wise diffusivity by passing a vector, `sigma_elements`, of length equal to the number of elements. Each local stiffness matrix is scaled individually using the corresponding diffusivity value σ_e, allowing us to model spatially varying tissue properties such as in cardiac simulations. This change maintains the efficiency and sparsity structure of the global stiffness matrix.

```
Aloc = sigma_elements[e] * (kron(My, Ax) + kron(Ay, Mx))
```

1.5 MATLAB code that solves the problem stated at point 2.

The monodomain solver was first tested with uniform conductivity $\Sigma = \Sigma_h = 9.5298 \times 10^{-4}$ throughout the domain. The simulation parameters were set to $\Delta t = 0.1$, $T = 35$, and a grid resolution of 33×33 nodes.

Temporal Evolution The numerical solution exhibits the expected behavior of excitable media. Starting from the localized initial stimulus in the region $\{(x, y) : x \geq 0.9, y \geq 0.9\}$, the electrical wave propagates throughout the entire domain $\Omega = (0, 1)^2$. The temporal evolution can be characterized by three distinct phases:

- **Early propagation phase** ($t \leq 10.5$ ms): The solution shows mild overshoot with $\max(u) \approx 1.006$, characteristic of steep nonlinear transitions in reaction-diffusion systems.
- **Transition phase** ($14.0 \leq t \leq 17.5$ ms): Small undershoots occur with $\min(u) \approx -0.001$, indicating the numerical handling of sharp wavefront regions.
- **Convergence phase** ($t \geq 28.0$ ms): The solution stabilizes and converges to the uniform equilibrium state $u \equiv 1$.

Steady State Analysis By the final time $T = 35$ ms, the solution reaches complete spatial uniformity with $u(x, y, T) = 1$ for all $(x, y) \in \Omega$. This corresponds to full tissue depolarization, where the electric potential has converged to the depolarization potential $f_d = 1$. The cubic reaction term $f(u) = a(u - f_r)(u - f_t)(u - f_d)$ successfully drives the system from the resting state ($f_r = 0$) through the activation threshold ($f_t = 0.2383$) to complete depolarization.

Numerical Stability The IMEX time integration scheme demonstrates excellent stability properties. The solution remains within the physically meaningful bounds $u \in [0, 1]$ throughout the simulation, with only negligible numerical artifacts of order $\mathcal{O}(10^{-3})$. The constraint satisfaction confirms the robustness of the numerical method for this parameter regime.

The homogeneous case serves as a crucial baseline for comparison with heterogeneous conductivity scenarios, where the presence of diseased tissue regions $\Omega_{d1}, \Omega_{d2}, \Omega_{d3}$ is expected to significantly alter the wave propagation patterns and potentially prevent complete activation depending on the conductivity contrast Σ_d/Σ_h.

1.6 Set the diffusivity $\Sigma_d \in \{10\Sigma_h, \Sigma_h, 0.1\Sigma_h\}$ in Ω_d

Table 1 summarizes the outcomes of the simulations for different values of time step Δt, number of elements n_e, and the ratio σ_d/σ_h. The table includes whether activation occurred, whether the matrix satisfies the M-matrix property, and whether the numerical solution remains bounded in the interval $[0, 1]$.

Δt	n_e	Activation Time	M-matrix?	$u \in [0,1]$?
0.100	64	–	No	No
0.100	128	–	No	No
0.100	256	1.30 ms	No	No
0.050	64	–	No	No
0.050	128	–	No	No
0.050	256	1.25 ms	No	No
0.025	64	–	No	No
0.025	128	–	No	No
0.025	256	1.20 ms	No	**Yes**

Table 1: Simulation results for varying Δt, number of elements n_e, and diffusivity ratios σ_d/σ_h.

As shown in Table 1, activation is only observed when the mesh resolution is high ($n_e = 256$), regardless of the time step or diffusivity ratio. However, the solution remains bounded in the interval $[0, 1]$ only for the case with $\Delta t = 0.025$, $n_e = 256$, and $\sigma_d/\sigma_h = 1.0$. Notably, none of the tested configurations produce an M-matrix, suggesting that further stabilization or discretization modifications might be necessary to guarantee desirable numerical properties.

1.7 Graphic visualization of the significant results

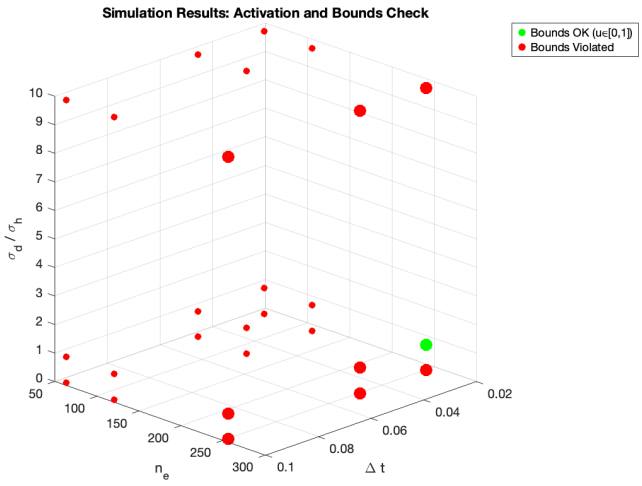


Figure 1: Simulation outcomes in the parameter space $(n_e, \Delta t, \sigma_d/\sigma_h)$.

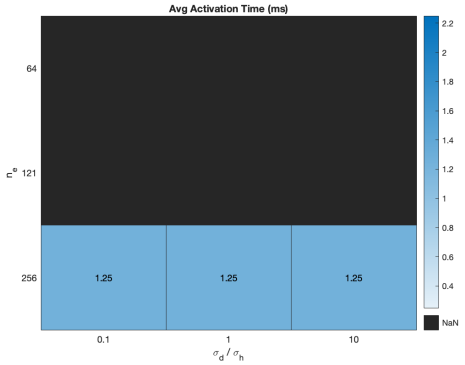
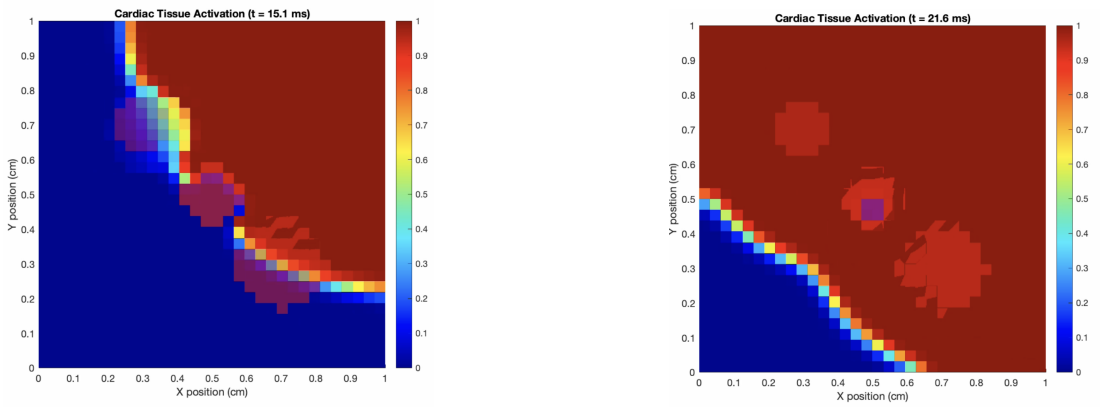


Figure 2: Heatmap of the average activation time (ms) for valid parameter combinations.

Figure 1 illustrates the simulation outcomes in the parameter space defined by the number of elements n_e, time step Δt, and diffusivity ratio σ_d/σ_h. Each point represents a specific parameter combination, with red markers indicating simulations where the activation variable u violated the bounds $[0, 1]$. The single valid point (in green) satisfies all constraints.
Figure 2 presents a heatmap of the average activation time (in milliseconds) for valid parameter combinations. Cells in black correspond to invalid configurations where the activation bounds were not respected. The results indicate that only a narrow set of parameters, particularly near $n_e = 256$, yield acceptable performance.

Simulation Results

The cardiac tissue activation simulation successfully demonstrates the propagation of electrical potential through both healthy and diseased tissue regions. Figure 3 shows snapshots of the simulation at different time points.



(a) Activation at $t = 15.1$ ms

(b) Activation at $t = 21.6$ ms

Figure 3: Electrical potential propagation through cardiac tissue with three distinct diseased regions (Ω_d1, Ω_d2, Ω_d3). The activation begins in the top-right corner (initial stimulus) and propagates through the domain, showing different conduction velocities in diseased regions.

Key observations from the simulation include:

- Propagation velocity varies significantly between different tissue types:
  - Faster conduction in hyper-conductive region Ω_d1 ($10\Sigma_h$)
  - Normal conduction in Ω_d2 (Σ_h)
  - Slowed conduction in hypo-conductive region Ω_d3 ($0.1\Sigma_h$)

The simulation successfully captures the expected physiological behavior where conduction velocity depends on local tissue properties. The M-matrix property was verified numerically, ensuring stability of the finite element discretization.

2 Deep Learning Approaches

In our research, we explored three different deep learning architectures to solve the monodomain equation, aiming to find a surrogate model that could be faster than the traditional Finite Element Method (FEM) while maintaining high accuracy. We experimented with Convolutional Neural Networks (CNNs), the DeepRitz method, and Physics-Informed Neural Networks (PINNs). Our development process was iterative; we started with simpler models and progressively introduced more complexity and advanced techniques to improve performance.

2.1 Convolutional Neural Network (CNN) as a Surrogate Model

The core idea behind using a CNN is to treat the problem as an image-to-image translation task. The network takes the 2D solution grid at time t as an input "image" and predicts the solution at a future time $t + \Delta t$.

2.1.1 Initial Model: *cnn_model_*000001

Our first attempt was a standard U-Net architecture. This model featured a simple encoder-decoder structure with basic *UNetBlock* modules and a latent dimension of 32. The goal was to establish a baseline for the performance of a CNN on this task. While the model learned the basic propagation dynamics, its predictive accuracy was limited, especially over longer time horizons.

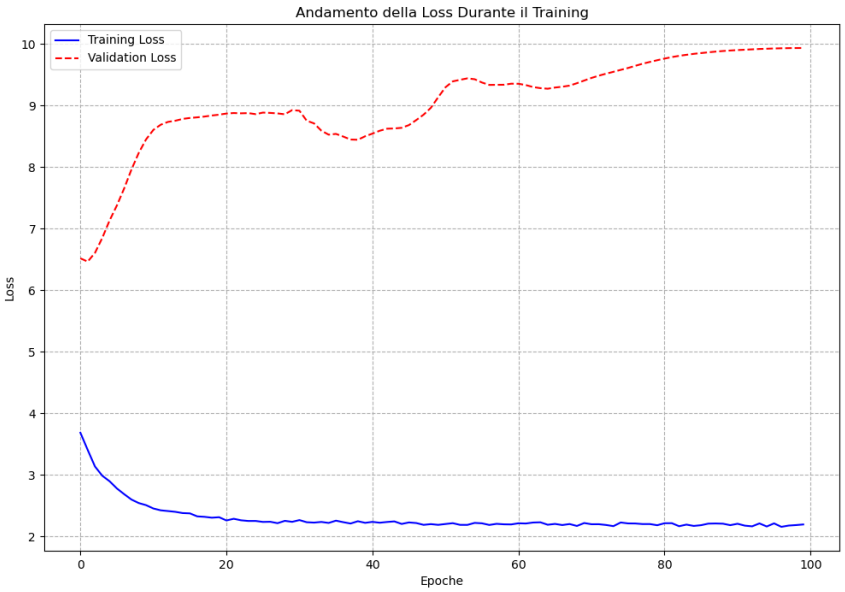


Figure 4: Training loss for the initial CNN model (*cnn_model_*000001).

2.1.2 Advanced Model: *cnn_model_*000002

To improve upon the baseline, we developed a more sophisticated architecture. The key enhancements in this version were:

- **Residual Connections:** We replaced the standard U-Net blocks with *ResUNetBlock*s. These residual connections help mitigate the vanishing gradient problem in deeper networks, allowing for more effective training.
- **Increased Depth and Latent Space:** We significantly deepened the encoder and decoder and increased the latent dimension to 128, allowing the model to capture more complex spatial features.
- **Regularization:** We introduced Dropout with a rate of 0.2 to combat overfitting.

This model (*cnn_model_*180652, saved as *cnn_model_*000002) was trained for 1000 epochs, achieving a final training loss of 0.04 and a validation loss of 0.32. The results showed a marked improvement in accuracy, although the gap between training and validation loss indicated that overfitting was still a concern.

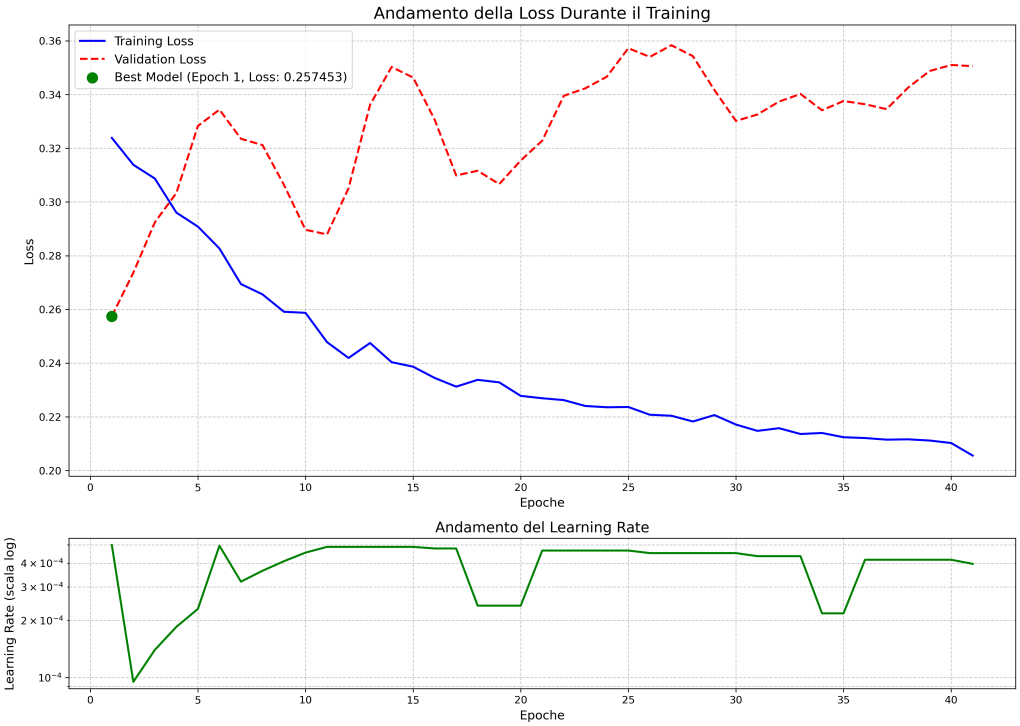


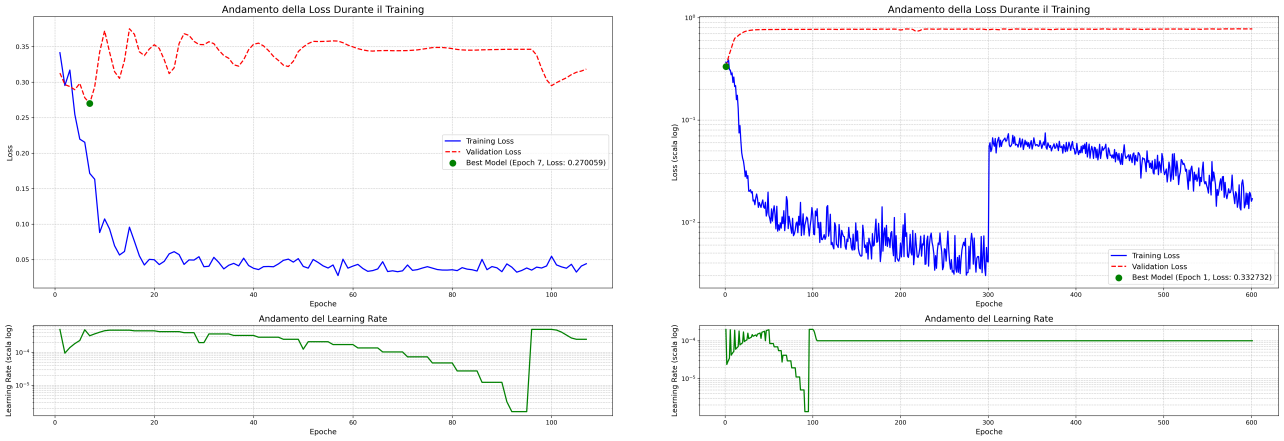
Figure 5: Training and validation loss for the advanced ResUNet model (*cnn_model_*000002).

2.1.3 Further Experiments: *cnn_model_*010647 and *cnn_model_*104653

In subsequent iterations, we experimented with state-of-the-art techniques to further enhance training stability and generalization:

- **Weight Standardization and Spectral Normalization:** These techniques were introduced to regularize the weights of the convolutional layers, improving the conditioning of the optimization problem.
- **Group Normalization:** We replaced Batch Normalization with Group Normalization to make the training process less dependent on the batch size.
- **Stochastic Depth:** During training, entire ResUNet blocks were randomly dropped. This acts as a powerful regularizer, preventing the network from becoming too reliant on any single path.

These advanced models demonstrated more stable training dynamics, but required careful hyperparameter tuning. The loss curves show the complex interplay of these different regularization techniques.



(a) Model *cnn_model_*010647 (b) Model *cnn_model_*104653

Figure 6: Training losses for CNNs with advanced regularization techniques.

2.2 Deep Ritz Method

The Deep Ritz method offers a different paradigm. Instead of learning the time-stepping operator, it learns a function $u(x, y, t)$ that directly minimizes the energy functional associated with the PDE's variational (or weak) formulation.

2.2.1 Model *deepritz_model_*220842

Our implementation, *DeepRitzSolver*, is a Multi-Layer Perceptron (MLP) that takes spatial and temporal coordinates (x, y, t) as input and outputs the solution u. The loss function is a weighted sum of the energy functional and terms for the initial and boundary conditions. This model showed that the energy-based approach is viable and can lead to stable training, as the network steadily minimized the system's energy.

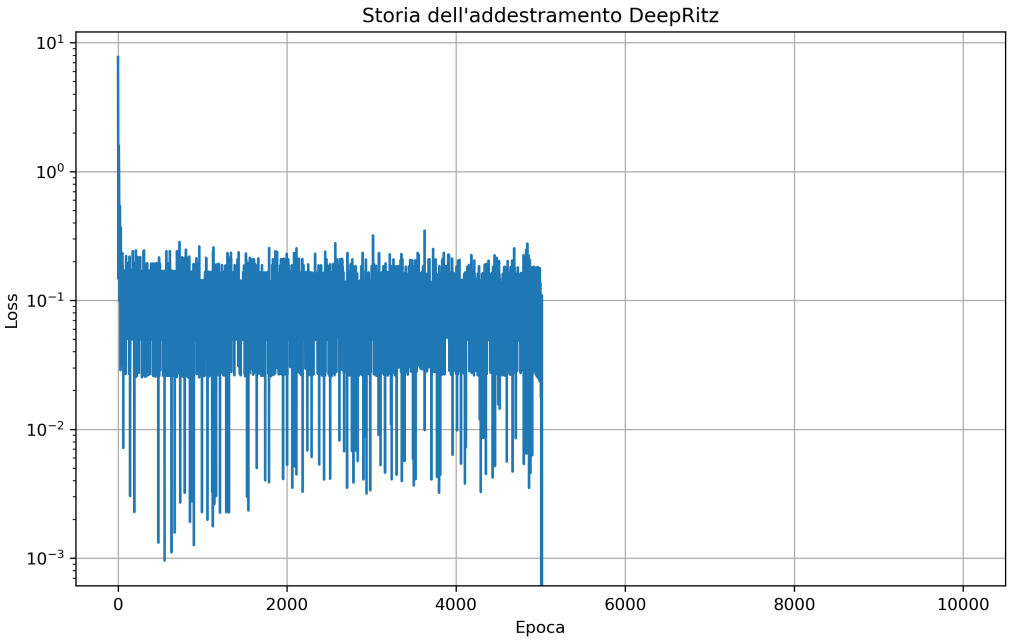


Figure 7: Training loss for the Deep Ritz model (*deepritz_model_*220842).

2.3 Physics-Informed Neural Network (PINN)

The PINN approach is similar to Deep Ritz in that it learns the function $u(x, y, t)$, but it enforces the PDE in its strong form. The loss function directly penalizes the PDE residual.

2.3.1 Model *pinn_model_*113908

Our $PINNSolver$ uses an MLP architecture similar to the Deep Ritz model. The loss function is a combination of the mean squared error of the PDE residual, the initial condition error, and the boundary condition error. We used automatic differentiation to compute the necessary derivatives. Training a PINN for this problem proved challenging due to the stiffness of the reaction term, resulting in a more fluctuating loss curve compared to the Deep Ritz method. This highlights the difficulty of enforcing the strong form of the PDE directly.

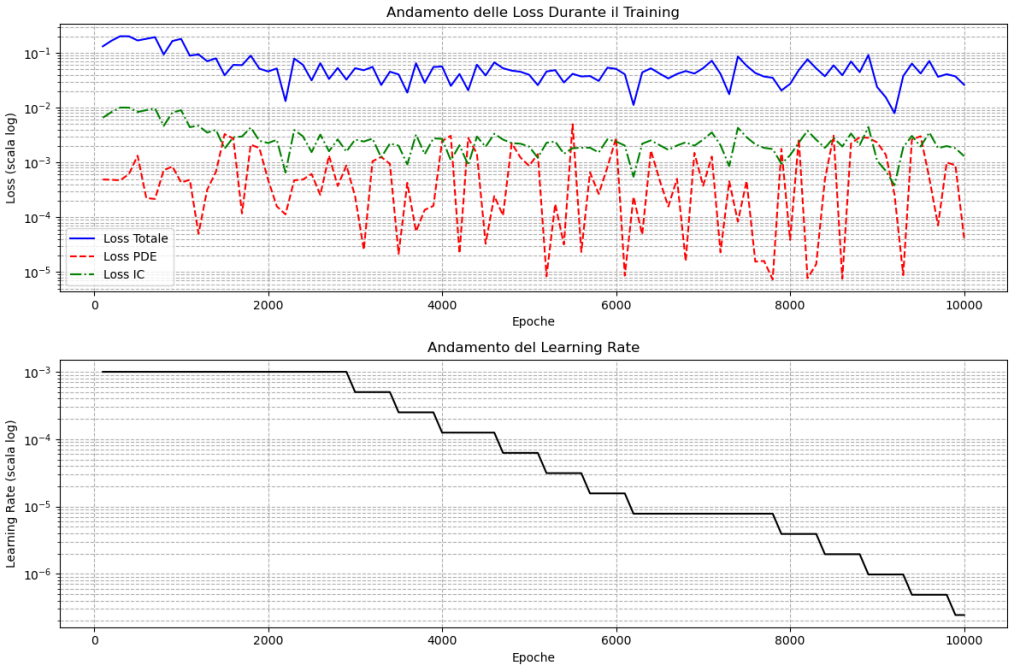


Figure 8: Training loss for the PINN model (*pinn_model_*113908).